



Advanced Farm House Security with IoT Dashboard

An IoT-based smart farm house security system that combines **Face Recognition, IoT Sensors, two ESP32 boards, an Arduino Uno, a Servo Motor, and a Live Web Dashboard**.

The system recognizes authorized faces using a laptop camera, controls the gate automatically, monitors temperature, humidity, and gas levels, and sends sensor data to a web dashboard.

Overview

The system uses a laptop webcam and a Python face-recognition application to identify authorized people.

When a recognized person is detected, the laptop sends an access command to **ESP32 #2 (Bridge)**. ESP32 #2 forwards the command over Wi-Fi to **ESP32 #1 (Main)**, which controls the gate Servo Motor.

At the same time, an **Arduino Uno** reads data from the DHT sensor and gas sensor and sends the readings to ESP32 #1. The Main ESP32 then sends the sensor data to the dashboard server through HTTP.

The overall system consists of:

- Laptop / PC with USB Webcam
- Python Face Recognition application
- ESP32 #1 — Main Controller
- ESP32 #2 — Communication Bridge
- Arduino Uno — Sensor Controller
- DHT11 / DHT22 — Temperature and Humidity Sensor
- MQ-2 / MQ-5 — Gas Sensor
- SG90 Servo Motor — Gate Controller
- Flask Dashboard Server
- Wi-Fi Router

System Components

Hardware

Component	Quantity	Purpose
Laptop / PC	1	Runs the Python face-recognition application
USB Webcam	1	Captures live video
ESP32 (Main)	1	Controls the gate, receives sensor data, and communicates with the dashboard

Component	Quantity	Purpose
ESP32 (Bridge)	1	Transfers commands from the laptop to the Main ESP32
Arduino Uno	1	Reads environmental sensors
DHT11 / DHT22	1	Measures temperature and humidity
MQ-2 / MQ-5 Gas Sensor	1	Detects gas/smoke level
SG90 Servo Motor, Laser, PIR	1	Opens and closes the gate
Wi-Fi Router	1	Provides the main local network
Dashboard Server	1	Receives and displays sensor data

Software

File	Language / Type	Purpose
<code>main.py</code>	Python	Main face-recognition application and command sender
<code>EncodeGenerator.py</code>	Python	Generates the face-encoding database
<code>EncodeFile.p</code>	Binary	Stores encoded faces
<code>Esp32WithArduinoAndDashboard.ino</code>	C++	Main ESP32 firmware
<code>Esp32withLaptop.ino</code>	C++	Bridge ESP32 firmware

How the System Works

1. Face Recognition

The laptop webcam continuously captures video.

The Python application:

1. Captures an image from the webcam.
2. Detects faces.
3. Compares detected faces with the stored face encodings.
4. Generates an access command.

Example commands:

87,1

indicates an authorized face.

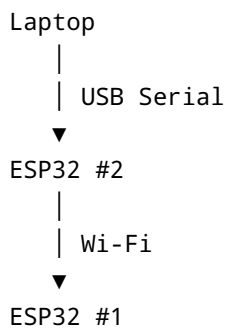
87,0

indicates an unauthorized face.

2. ESP32 #2 — Bridge

ESP32 #2 receives the command from the laptop through USB Serial.

The Bridge ESP32 then forwards the command to ESP32 #1 through its own Wi-Fi Access Point.



The communication uses:

SSID: No_Network
IP: 192.168.4.1
Port: 80

3. ESP32 #1 — Gate Control

ESP32 #1 receives and parses the command.

For:

87,1

the Main ESP32 sets the Servo Motor to:

90°

The gate opens and remains open for approximately five seconds.

After five seconds:

Servo → 0°

and the gate closes.

For:

87,0

the gate remains closed.

4. Arduino Uno — Sensor Monitoring

The Arduino Uno continuously reads the connected sensors.

Example sensor output:

Humidity: 65, Temperature: 28, Gas Sensor: 150

The Arduino sends this data to ESP32 #1 using serial communication.

```
Arduino Uno
  |
  | UART
  ▼
ESP32 #1
```

5. Dashboard Communication

ESP32 #1 sends sensor data to the Flask dashboard using an HTTP POST request.

Endpoint:

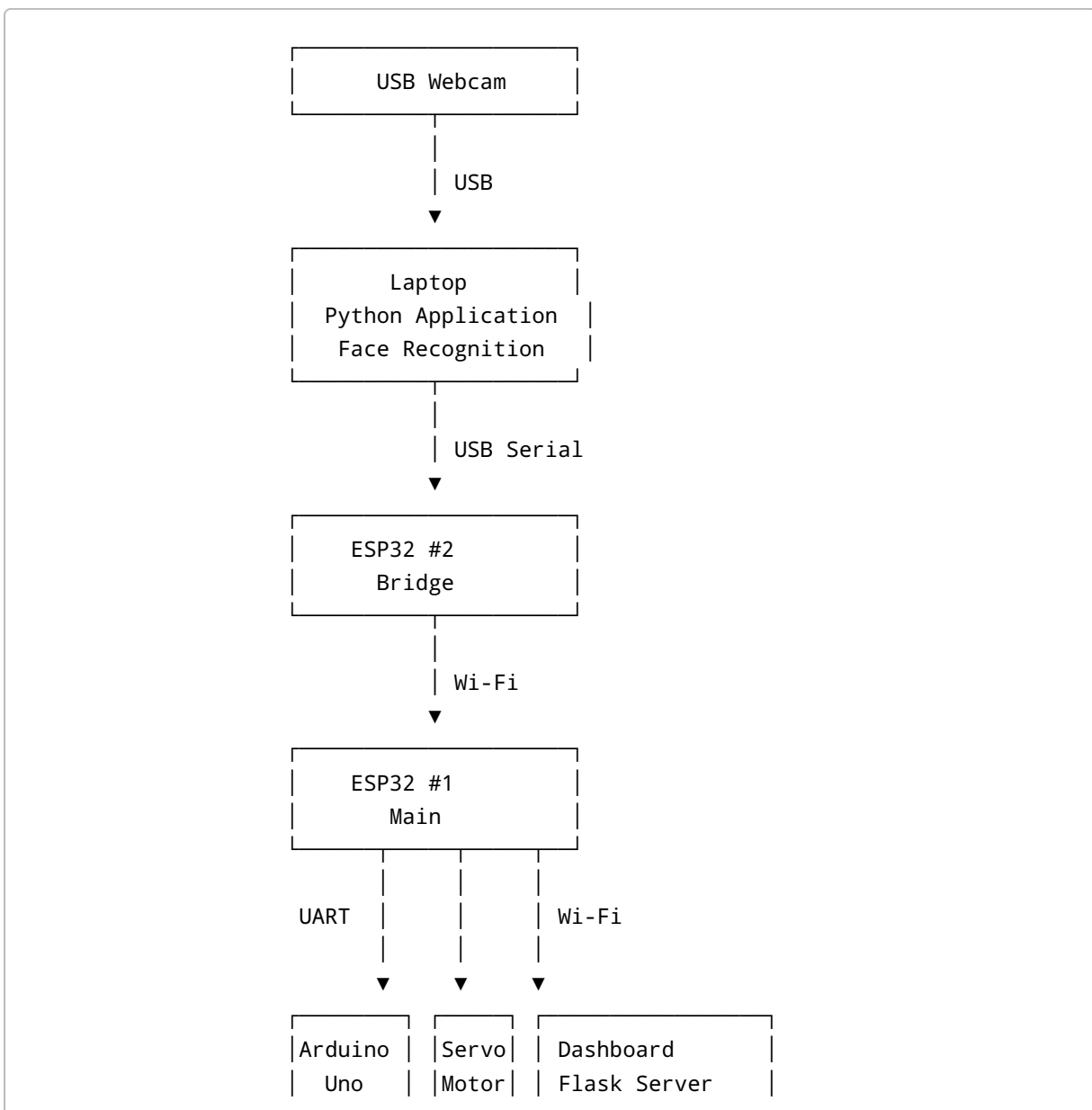
POST /sensor-data

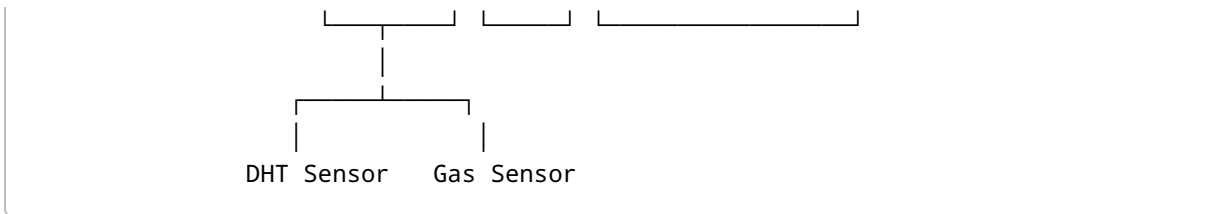
Example request:

```
{  
  "writeApiKey": "YOUR_API_KEY",  
  "sensorName": "DHT",  
  "value": 150  
}
```

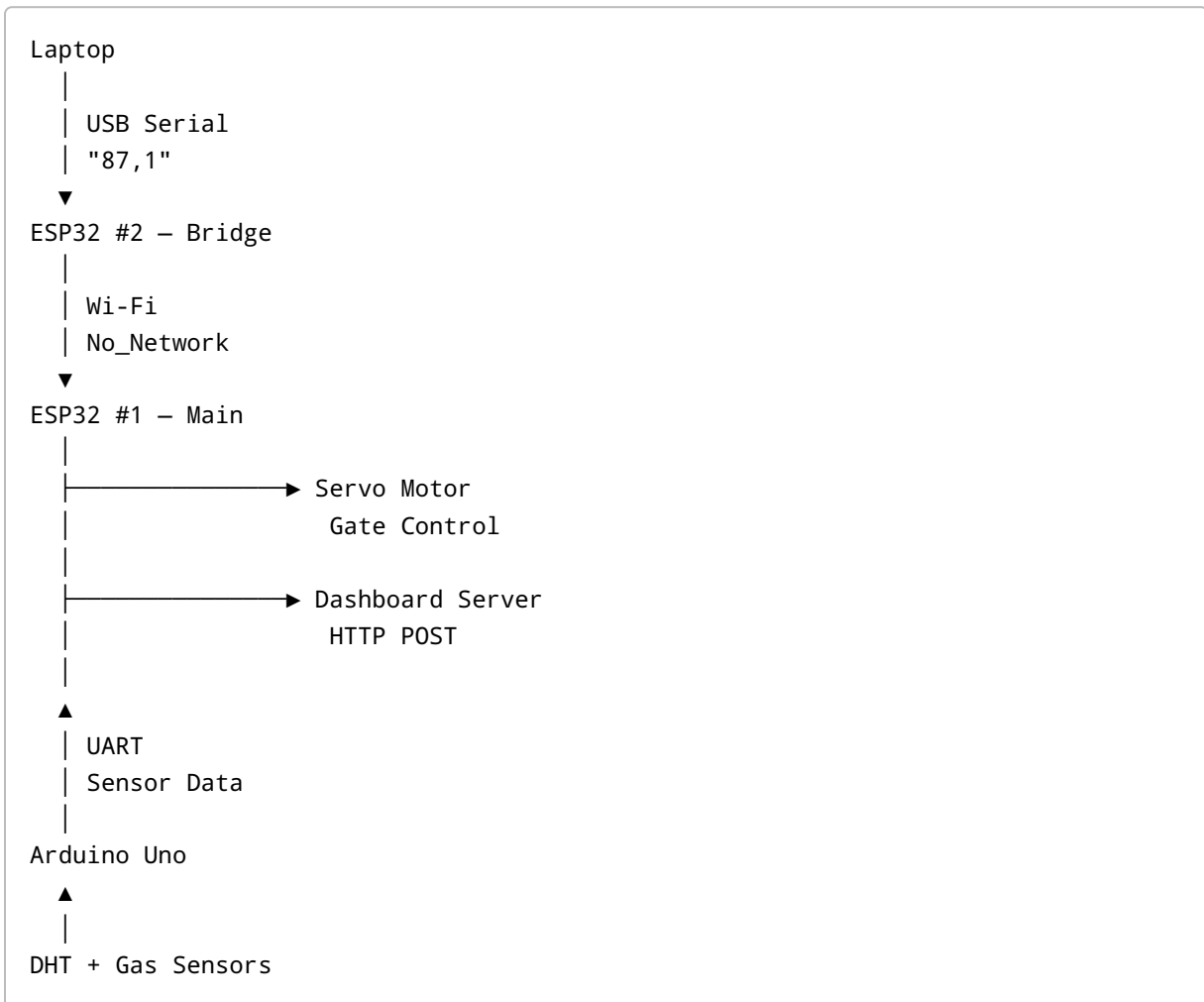
The dashboard server receives the data and updates the monitoring interface.

System Architecture





Communication Flow



The system uses two separate wireless connections:

- **ESP32 #2 → ESP32 #1:** ESP32 #1 provides the **No_Network** Access Point.
- **ESP32 #1 → Dashboard:** ESP32 #1 connects to the main Wi-Fi network, such as **Jalish**.

Data Formats

Laptop → ESP32 #2

```
"value1,value2\n"
```

Example:

```
87,1
```

ESP32 #2 → ESP32 #1

```
"value1,value2\n"
```

Example:

```
87,1
```

Arduino Uno → ESP32 #1

```
"Humidity: X, Temperature: Y, Gas Sensor: Z\n"
```

Example:

```
Humidity: 65, Temperature: 28, Gas Sensor: 150
```

ESP32 #1 → Dashboard

Data is transmitted using HTTP POST with JSON.

```
{  
  "writeApiKey": "YOUR_API_KEY",  
  "sensorName": "DHT",  
  "value": 150  
}
```

Project Structure

```
Advanced-Farm-House-security-With-IoT-Dashboard/
├── Esp32WithDashboardAndArduino/
│   └── Esp32WithArduinoAndDashboard.ino
├── Esp32withLaptop/
│   └── Esp32withLaptop.ino
├── SensorProject/
│   ├── main.py
│   ├── Duplicate.py
│   ├── EncodeGenerator.py
│   ├── video.py
│   ├── test.py
│   ├── EncodeFile.p
│   ├── Images/
│   ├── Files/
│   ├── Files.zip
│   └── Resources/
│       ├── background.png
│       └── Modes/
└── README.md
```

Hardware Wiring

Arduino Uno → ESP32 #1

Arduino Uno		ESP32 #1
TX (Pin 1)	→	GPI016 (RX2)
RX (Pin 0)	←	GPI017 (TX2)
GND	→	GND

Servo Motor → ESP32 #1

Servo Motor		ESP32 #1
Signal	→	GPI05

Power → 5V / 3.3V
GND → GND

ESP32 #2 → Laptop

ESP32 #2 Bridge Laptop

USB Port → USB Port

Main ESP32 Pin Map

ESP32 Pin	Function	Connected To
GPIO16	Serial2 RX	Arduino TX
GPIO17	Serial2 TX	Arduino RX
GPIO5	Servo PWM	SG90 Signal
GND	Ground	Arduino and Servo GND
5V / 3.3V	Power	Servo Power

Installation and Setup

1. Install Python Dependencies

```
pip install opencv-python face_recognition numpy pyserial requests cmake dlib
```

2. Generate the Face Database

Place images of authorized people inside:

SensorProject/Images/

Use the person's name as the image filename.

Example:

rifat.jpg

Then run:

```
cd SensorProject
python EncodeGenerator.py
```

This generates:

```
EncodeFile.p
```

which is used by `main.py`.

3. Configure Arduino Uno

Install the following libraries through the Arduino IDE:

- DHT sensor library by Adafruit
- Adafruit Unified Sensor

Upload the sensor sketch to the Arduino Uno.

The expected serial output is similar to:

```
Humidity: 65, Temperature: 28, Gas Sensor: 150
```

4. Configure Main ESP32

Open:

```
Esp32WithDashboardAndArduino/Esp32WithArduinoAndDashboard.ino
```

Update the Wi-Fi and dashboard configuration before uploading:

```
const char* ssid      = "YOUR_WIFI_NAME";
const char* password   = "YOUR_WIFI_PASSWORD";
const char* ap_ssid    = "No_Network";
const char* ap_password = "YOUR_AP_PASSWORD";
const char* serverName = "http://YOUR_DASHBOARD_IP:5000/sensor-data";
const char* writeApiKey = "YOUR_API_KEY";
```

Select:

Board: ESP32 Dev Module

and upload the firmware.

5. Configure Bridge ESP32

Open:

Esp32withLaptop/Esp32withLaptop.ino

Configure the connection to the Main ESP32:

```
const char* ssid      = "No_Network";  
const char* password  = "YOUR_AP_PASSWORD";  
const char* serverIP  = "192.168.4.1";  
const int  serverPort = 80;
```

Select:

Board: ESP32 Dev Module

and upload the firmware.

6. Connect the Hardware

Arduino TX	→ ESP32 #1 GPIO16
Arduino RX	→ ESP32 #1 GPIO17
Arduino GND	→ ESP32 #1 GND
ESP32 #1 GPIO5	→ Servo Signal
ESP32 #2 USB	→ Laptop USB

7. Run the Python Application

```
cd SensorProject  
python main.py
```

The system is now ready to operate.

Configuration

Python Serial Port

For Windows:

```
ser = serial.Serial('COM3', 115200)
```

For Linux/macOS:

```
ser = serial.Serial('/dev/ttyUSB0', 115200)
```

Main ESP32

```
// Main Wi-Fi
const char* ssid      = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

// ESP32 #1 Access Point
const char* ap_ssid    = "No_Network";
const char* ap_password = "YOUR_AP_PASSWORD";

// Serial2
#define RX2 16
#define TX2 17

// Servo
myServo.attach(5);

// Dashboard
const char* serverName = "http://YOUR_DASHBOARD_IP:5000/sensor-data";
const char* writeApiKey = "YOUR_API_KEY";
```

Bridge ESP32

```
const char* ssid      = "No_Network";
const char* password = "YOUR_AP_PASSWORD";
const char* serverIP  = "192.168.4.1";
const int  serverPort = 80;
```

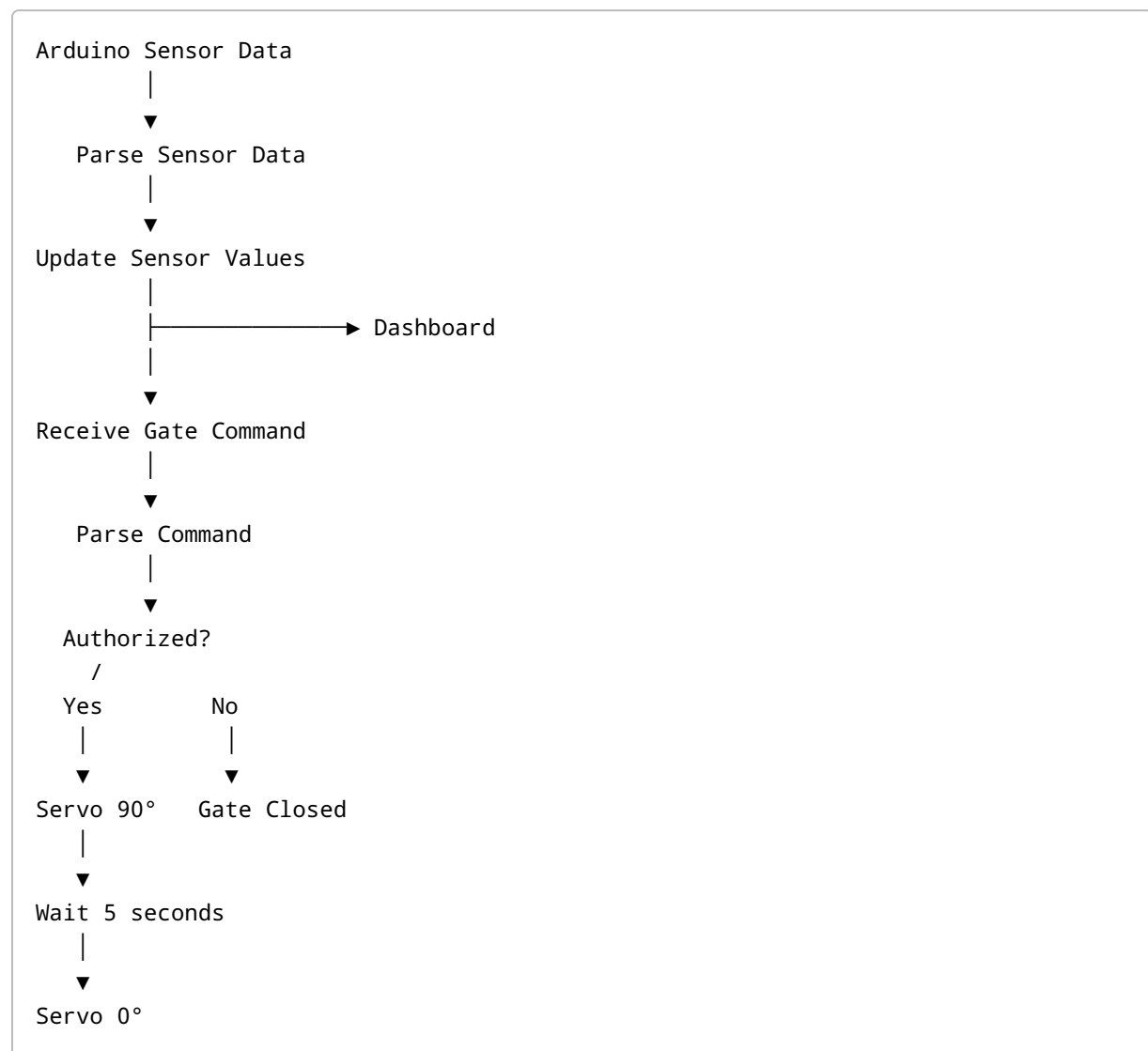
Code Flow

Main ESP32

The Main ESP32 performs four primary tasks:

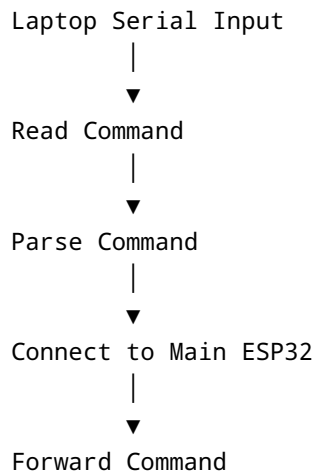
1. Reads sensor data from the Arduino Uno.
2. Receives access commands from ESP32 #2.
3. Controls the Servo Motor based on the access command.
4. Sends sensor data to the dashboard using HTTP POST.

Conceptually:



Bridge ESP32

The Bridge ESP32 continuously checks for commands from the laptop.



API Reference

POST /sensor-data

Receives sensor data from ESP32 #1.

Headers

Content-Type: application/json

Request Body

```
{
  "writeApiKey": "YOUR_API_KEY",
  "sensorName": "DHT",
  "value": 150
}
```

Success Response

```
{
  "status": "success"
}
```

Troubleshooting

`EncodeFile.p` not found

The face database has not been generated.

Run:

```
python EncodeGenerator.py
```

Serial Port Error

Check the correct COM port in Windows Device Manager and update the port in `main.py`.

ESP32 #1 Cannot Connect to Wi-Fi

Check the configured Wi-Fi SSID and password.

ESP32 #2 Cannot Connect

Make sure ESP32 #1 is powered on and its `No_Network` Access Point is running.

Servo Does Not Move

Check that the received command is being parsed correctly and verify the Servo signal connection to GPIO5.

Dashboard Does Not Receive Data

Check the dashboard server IP and make sure the endpoint is reachable:

```
http://YOUR_DASHBOARD_IP:5000/sensor-data
```

Arduino Data Parsing Fails

Verify that the Arduino sends data using the expected format:

```
Humidity: 65, Temperature: 28, Gas Sensor: 150
```

Face Recognition Fails

Update the images in:

```
SensorProject/Images/
```

and regenerate:

```
EncodeFile.p
```

Dependencies

Python

```
pip install opencv-python face_recognition numpy pyserial requests cmake dlib
```

Main libraries:

- `opencv-python` — webcam and image processing
- `face_recognition` — face detection and encoding
- `numpy` — numerical processing
- `pyserial` — serial communication
- `requests` — HTTP communication
- `dlib` — backend used by `face_recognition`
- `cmake` — required for building certain dependencies

Arduino / ESP32

- `WiFi.h` — Wi-Fi Station and Access Point functionality
- `HTTPClient.h` — HTTP communication with the dashboard
- `ESP32Servo.h` — Servo Motor control
- `HardwareSerial` — UART communication with Arduino Uno
- `DHT.h` — DHT11/DHT22 sensor readings

Quick Start

```
# 1. Generate the face database
cd SensorProject
python EncodeGenerator.py

# 2. Upload the sensor firmware to Arduino Uno

# 3. Configure and upload the Main ESP32 firmware
#   Esp32WithArduinoAndDashboard.ino

# 4. Configure and upload the Bridge ESP32 firmware
#   Esp32withLaptop.ino
```

```
# 5. Connect the hardware
#   Arduino TX      → ESP32 #1 GPIO16
#   Arduino RX      → ESP32 #1 GPIO17
#   Arduino GND     → ESP32 #1 GND
#   ESP32 #1 GPIO5  → Servo Signal
#   ESP32 #2 USB    → Laptop

# 6. Start the Python application
python main.py
```

License

This project is open source and available under the MIT License.

Network Requirements

The **Laptop**, **ESP32 #1**, and **Dashboard Server** should be connected to the same main Wi-Fi network.

ESP32 #2 does not connect directly to this main network. Instead, it connects to the dedicated Access Point created by ESP32 #1:

```
Main Wi-Fi Network
├─ Laptop
├─ ESP32 #1
└─ Dashboard Server

ESP32 #1 Access Point
└─ ESP32 #2
```

This separation allows ESP32 #2 to receive commands from the laptop through USB Serial and forward them to ESP32 #1 over its dedicated wireless connection.